PERTEMUAN 5 · Praktikum 1

Implementasi Lexer Hand-Written dan Flex

Universitas Bale Bandung

Fakultas Teknologi Informasi · Program Studi Teknik Informatika

Mata Kuliah Teknik Kompilasi (MWCA017) · Semester Ganjil · 3 SKS

Dosen: Cecep Suwanda, S.Si., M.Kom.

Capaian Pertemuan

CPMK induk: CPMK 02 — Memahami konsep Otomata Hingga (*Finite Automata*).

Sub-CPMK yang dinilai

Sub-CPMK 2.4: Mengimplementasikan *lexer* subset bahasa C dalam dua versi, yaitu *hand-written* dan Flex, lalu menguji keluarannya pada berkas masukan uji.

Cakupan Praktikum 1

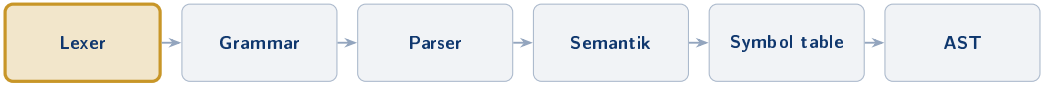
Praktikum 1 menilai Sub-CPMK 2.1–2.4. Artefak 2.1 (tabel ekspresi reguler), 2.2 (simulator NFA/DFA), dan 2.3 (tabel konversi dan minimalisasi) dari Pertemuan 3–4 ikut dikumpulkan bersama lexer.

Agenda Pertemuan (150 menit)

Menit	Kegiatan
20	Briefing praktikum, cakupan artefak, dan rubrik
60	Praktik: lexer *hand-written* dalam C
40	Praktik: lexer Flex dan perbandingan aturan
20	Pengujian pada berkas masukan yang sama
10	Pemeriksaan keluaran, laporan, dan penutup

Porsi praktik 120 menit. Dosen mendampingi proses implementasi dan *debugging*; setiap mahasiswa menyerahkan artefak pada akhir pertemuan.

Posisi Pertemuan pada Rantai Proyek



- Pertemuan ini menutup simpul **lexer** dan menghasilkan Milestone M1.
- Dua implementasi diuji pada berkas masukan yang sama agar perilakunya dapat dibandingkan.
- Keluaran lexer menjadi masukan parser pada Pertemuan 6 dan seterusnya.

Arsitektur Scanner: Buffer dan Sentinel



- Lexer membaca blok besar ke memori, bukan satu karakter dari disk.
- Sentinel `EOF` di akhir setiap buffer memicu pengisian buffer berikutnya.
- `lexeme_begin` menandai awal lexeme; `forward` bergerak selama karakter cocok.

Lookahead dan Kompleksitas

- Lookahead melihat karakter berikutnya tanpa mengonsumsinya.
- Satu karakter cukup untuk membedakan < dan <=.
- Komentar blok dan string escape memerlukan lookahead lebih panjang.
- Implementasi menyediakan `peek()` dan `peek_next()`.

Kompleksitas

Setiap karakter diproses paling banyak dua kali, sehingga waktu $O(n)$. Ruang $O(b)$ untuk buffer berukuran b, ditambah tabel transisi DFA. Pada proyek Subset-C, seluruh berkas dibaca ke memori.

Struktur Data Lexer (lexer.h)

```c
typedef struct {
    const char *source;
    size_t length;
    size_t pos;
    int line;
    int column;
    const char *filename;
    int had_error;
} Lexer;

void lexer_init(Lexer *lexer, const char *source,
     const char *filename);
Token lexer_next(Lexer *lexer);
void token_print(FILE *out, const Token *token);
```

Listing 1: Struktur Lexer pada lexer.h

- source dan length: isi berkas.
- pos: posisi baca saat ini.
- line dan column: lokasi token.
- had_error: penanda kesalahan.
- Pemisahan header dan implementasi memudahkan pengujian.

Operasi Dasar Scanner

Tiga operasi fondasi

`peek()` mengembalikan karakter saat ini; `peek_next()` mengembalikan karakter berikutnya; `advance()` memajukan posisi dan memperbarui baris serta kolom.

- Ketiganya menangani akhir masukan dengan mengembalikan `EOF`.
- Semua operasi lain dibangun di atas ketiganya.
- Logika pelacakan lokasi ditulis satu kali.
- `skip_space` membuang spasi, tab, baris baru, dan komentar.

Alur Fungsi `lexer_next`

1. Lewati spasi dan komentar.
2. Catat baris, kolom, dan posisi awal.
3. Jika akhir masukan, kembalikan `TOK_EOF`.
4. Jika huruf atau garis bawah, pindai identifier lalu periksa tabel kata kunci.
5. Jika digit, pindai bilangan bulat atau pecahan.
6. Jika kutip, pindai char atau string.
7. Jika operator, periksa operator dua karakter lalu satu karakter.
8. Jika tidak dikenal, hasilkan `TOK_ERROR`.

Cabang Identifier pada `lexer.c`

```
166     if (isalpha((unsigned char)ch) || ch == '_') {
167         TokenType type;
168
169         advance(lexer);
170         while (isalnum((unsigned char)peek(lexer))
                ||
171                peek(lexer) == '_') {
172             advance(lexer);
173         }
174         type = keyword_type(lexer->source + start,
175                             (int)(lexer->pos -
                                   start));
176         return make_token(lexer, type, start, line
                , column);
```

Listing 2: Pemindaian identifier dan pemeriksaan kata kunci

- `isalpha` atau _ memulai identifier.
- `isalnum` atau _ melanjutkan lexeme.
- `keyword_type` memeriksa tabel kata kunci.
- Jika tidak cocok, lexeme menjadi `TOK_IDENT`.
- Daftar kata kunci terpisah dari logika scanning.

Kata Kunci dan Operator Dua Karakter

Kata kunci

Lexer memindai identifier terlebih dahulu, lalu membandingkan lexeme dengan tabel kata kunci. Menambah kata kunci cukup mengubah tabel, bukan alur scanning.

Tanpa urutan ini, <= akan dipecah menjadi < dan =.

Operator dua karakter

Operator seperti <=, >=, ==, !=, &&, dan || diperiksa sebelum operator satu karakter. Setelah karakter pertama dikonsumsi, `peek()` menentukan apakah karakter kedua melanjutkan operator.

Struktur Berkas Flex

1. **Definisi**: deklarasi, makro, dan kode C.
2. **Aturan**: pasangan pattern dan aksi, dipisahkan `%%`.
3. **Kode pengguna**: fungsi tambahan dan `main`.

Cara kerja

Flex mengubah aturan menjadi DFA dan menghasilkan fungsi `yylex()` dalam berkas C. Pattern ditulis dengan ekspresi reguler; aksi ditulis dalam C. Aturan Flex memilih pattern terpanjang; bila panjangnya sama, aturan yang lebih awal menang.

Aturan Lexer Flex

```
28 DIGIT    [0-9]
29 ID       [A-Za-z_][A-Za-z0-9_]*
30
31 %%
32
33 [ \t\r\n]+                { /* abaikan */ }
34
35 "//"[^\n]*                { /* komentar baris */ }
36
37 "/*"([^*]|\*+[^*/])*\*+"/" { /* komentar blok */ }
38
39 "int"                      { return TOK_INT; }
40 "float"                    { return TOK_FLOAT; }
41 "char"                     { return TOK_CHAR; }
42 "void"                     { return TOK_VOID; }
```

Listing 3: Definisi dan aturan awal `lexer_flex.l`

- Makro `DIGIT` dan `ID` membuat aturan ringkas.
- Whitespace dan komentar tidak menghasilkan token.
- Kata kunci ditulis sebelum identifier.
- Aksi mengembalikan jenis token; lexeme diambil dari `yytext`.

Lexer Tangan dibandingkan Lexer Flex

Aspek	Lexer tangan	Lexer Flex
Kontrol	Penuh atas setiap detail	Terbatas pada aksi
Keterbacaan	Panjang, tetapi eksplisit	Ringkas dan deklaratif
Kualitas kesalahan	Dapat disesuaikan	Perlu kode tambahan
Perubahan pattern	Mengubah kode dan menguji ulang	Mengubah aturan
Ketergantungan	C standar	Flex dan generator
Kinerja	Bergantung implementasi	DFA yang dioptimalkan

Keduanya diimplementasikan agar mahasiswa memahami *trade-off* antara kontrol dan produktivitas.

Penanganan Kesalahan Leksikal

- Karakter ilegal, misalnya `@` atau `#`.
- String tidak ditutup.
- Char kosong atau terlalu panjang.
- Komentar blok tidak ditutup.
- Float tanpa digit setelah titik.

Pemulihan

Hasilkan `TOK_ERROR`, catat lokasi, lewati satu karakter atau sampai batas aman, lalu lanjutkan scanning. Lexer tidak berhenti pada kesalahan pertama.

Pesan menyebut lokasi dan jenis masalah, misalnya `baris 1, kolom 10: digit diharapkan setelah titik.`

Pelaporan Kesalahan (error.c)

```
4
5  void error_at(const char *filename, int line,
       int column,
6                const char *message) {
7      fprintf(stderr, "%s:%d:%d: error: %s\n",
8              filename ? filename : "<input>",
                   line, column, message);
9  }
```

Listing 4: Fungsi error_at pada error.c

- Format: berkas:baris:kolom: error: pesan.
- Mudah dibaca manusia dan diurai alat.
- Editor dapat melompat ke lokasi kesalahan.
- Hindari pesan generik seperti error atau invalid token.

Contoh: Menelusuri `a <= b;`

1. `skip_space`; posisi awal 1:1.
2. `a` → `IDENT(a)`.
3. Spasi dilewati; `<` dibaca, `peek()` melihat `=`.
4. `<=` cocok → `LE`; posisi maju satu karakter lagi.
5. `b` → `IDENT(b)`.
6. `;` → `SEMI`.
7. Akhir masukan → `EOF`.

Keluaran: `1:1 IDENT a, 1:3 LE <=, 1:6 IDENT b, 1:7 SEMI ;`.

Contoh: Pemulihan pada `int a = 3.;`

Token yang dihasilkan

`INT`, `IDENT(a)`, `ASSIGN`, `INT_LIT(3)`.

Kesalahan

Titik tanpa digit berikutnya tidak dimasukkan ke bilangan. Pemanggilan berikutnya menghasilkan `TOK_ERROR`: `baris 1, kolom 10: karakter tidak dikenal`.

- Lexer melanjutkan ke `;`.
- Parser menerima token kesalahan dan dapat melaporkan masalah tanpa kehilangan sisa program.
- Lokasi awal token berikutnya tetap dihitung dengan benar.

Pengujian Kedua Lexer

- `basic.sc`: program minimal.
- `operators.sc`: operator dan tanda baca.
- `literals.sc`: integer, float, char, string.
- `errors.sc`: karakter ilegal dan literal tidak lengkap.

Format keluaran

Setiap baris: `BARIS:KOLOM JENIS LEXEME`.
Bandingkan dengan berkas `.expected`.
Skrip `run-tests.bat` menjalankan lexer, membandingkan keluaran, dan melaporkan tes yang gagal.

Kedua lexer diuji pada berkas masukan yang sama; keluarannya harus identik, kecuali perbedaan penanganan baris baru yang didokumentasikan.

Kegiatan Praktikum: M1

Langkah kerja

1. Lengkapi `include/token.h` dan `include/lexer.h`.
2. Implementasikan `src/lexer.c` dan `src/error.c`.
3. Implementasikan `src/main_lex.c`.
4. Implementasikan `src/lexer_flex.l`.
5. Jalankan `build.bat` dan `run-tests.bat`.
6. Bandingkan keluaran lexer tangan dan Flex.
7. Tulis laporan singkat dan serahkan.

Kumpulkan juga tabel ekspresi reguler (2.1), simulator NFA/DFA (2.2), serta tabel konversi dan minimalisasi (2.3) dari Pertemuan 3–4.

Rubrik Praktikum 1

Aspek	Indikator	Bobot
Kebenaran	Semua token dikenali; keluaran kedua lexer sesuai berkas harapan	40%
Kualitas kode	Struktur, penamaan, komentar, dan tanpa peringatan kompilasi	25%
Kelengkapan	Lexer tangan dan Flex, 30 tes, laporan	20%
Pengujian	Kasus normal, batas, dan kesalahan terdokumentasi	15%

Rubrik ini sama dengan rubrik artefak praktikum pada bagian 10 RPS.

Tugas dan Artefak

Artefak Sub-CPMK 2.4

Lexer *hand-written* dan lexer Flex, diuji pada berkas masukan yang sama, disertai keluaran program dan laporan.

- Dua lexer berjalan dan menghasilkan aliran token setara.
- Minimal 30 kasus uji dengan berkas `.expected`.
- Diagnostik menyebut lokasi dan penyebab.
- Artefak 2.1–2.3 dari Pertemuan 3–4 ikut dikumpulkan karena Praktikum 1 menilai 2.1–2.4.
- Laporan memuat desain, hasil tes, kendala, dan refleksi.

Praktikum 1 berbobot 12% dan dilaporkan setelah Milestone M1 selesai.

Rangkuman dan Jembatan

- Buffer, sentinel, `lexeme_begin`, dan `forward` membuat scanner bekerja dalam $O(n)$.
- Lexer tangan memberi kontrol penuh; lexer Flex ringkas dan deklaratif.
- Keduanya memakai aturan yang sama: operator panjang lebih dulu, float sebelum integer, kata kunci sebelum identifier.
- Kesalahan leksikal menghasilkan `TOK_ERROR` dan diagnostik yang menyebut lokasi.
- Praktikum 1 menilai Sub-CPMK 2.1–2.4 melalui artefak lexer, tabel regex, simulator, dan tabel konversi.

Pertemuan berikutnya

Pertemuan 6 beralih dari token ke tata bahasa bebas konteks: BNF/EBNF, derivasi, dan pohon sintaks. Aliran token dari lexer menjadi masukan parser.